NetTalk ARMi Fingerprint Login

Fingerprint, PIN or screen-lock sign-in (WebAuthn / passkeys) for NetTalk web servers

Developer manual

Template installation, what the installer writes, and how to add it step by step to a NetTalk Server application.

Document version: 1.0 (draft)

Date: October 1, 2026

Tested with: Clarion 11.1 and NetTalk, on PC (Chrome / Edge with Windows Hello) and on Android (Chrome) over HTTPS.

Contents

1. What NetTalk ARMi Login is 4
1.1 What it includes 4
1.2 What it does not do 4
1.3 Requirements 4
2. Process overview 4
2.1 The passkey code: how the data travels 5
3. What the installer does 6
3.1 Files that are copied 6
3.2 What the template does when the application is generated 6
3.3 What the installer does NOT touch 7
3.4 What the template and the application write afterwards 7
4. Adding the template to a NetTalk Server application 8
Step 1. Check that the template is registered 8
Step 2. The browser script 8
Step 3. Add the global extension 9
Step 4. Login procedure extension 9
Step 5. User session code (embed) 10
Step 6. Offer extension on the destination page 10
Step 7. Texts and translation 11
Step 8. Generate and compile 11
Step 9. Test 11
Variant: Multi-DLL program 12
5. Extension reference 12
5.1 NetTalk ARMi Login - activate (global) 12
5.2 NetTalk ARMi Login - fingerprint button on the login form 13
5.3 NetTalk ARMi Login - offer to activate the fingerprint after login 14
6. How it works internally 15
6.1 Activating a fingerprint 15
6.2 Signing in with the fingerprint 15
6.3 What the server verifies 15
6.4 Browser - server communication 15
7. Going to production 16
7.1 Server behind a proxy or forwarding 16
8. Sample application 16
8.1 How to use it 17
9. Troubleshooting 17
9.1 Class error codes 17

10. Frequently asked questions..... 18
Appendix A. Glossary..... 19

If the table of contents appears empty, right-click it and choose "Update Field".

1. What NetTalk ARMi Login is

NetTalk ARMi Login is a set of templates and a Clarion class that add to a **NetTalk Server** application sign-in with the user's fingerprint, PIN or device screen lock, using the **WebAuthn** standard (also known as "passkeys").

The user signs in once with user name and password, the application offers to activate the fingerprint on that device and, from then on, the user can sign in by tapping a button and confirming with the fingerprint or PIN, without typing anything.

1.1 What it includes

- **A class** (`NetWebAuthnClass`) that creates challenges, verifies ES256 signatures with Windows CNG (`bcrypt.dll`) and stores the credentials in a TopSpeed file.
- **Three extensions** (in a single template): a global one, one for the login procedure and one for the page the user lands on after signing in.
- **A browser script** (`NetTalk_ARMiLogin.js`) that talks to the device and to the server.
- **An installer** that puts everything in the Clarion folders (chapter 3).

1.2 What it does not do

- It does not store fingerprints, PINs, private keys or passwords. The fingerprint never leaves the device; the server only stores the **public key**.
- It does not replace user-and-password login: it complements it. Both keep working.
- It does not depend on external services or other languages: everything runs inside the Clarion server.

1.3 Requirements

Requirement	Detail
Development environment	Clarion 10, 11.x or 12 with NetTalk installed. The application must be a **NetTalk Server** with a login procedure built with the NetWebForm template.
Production server	Windows 7 / Server 2008 R2 or later (it uses `bcrypt.dll`, which is part of Windows).
HTTPS and domain	WebAuthn **only works in a secure origin**: HTTPS with a valid certificate, or `http://localhost`. In addition, the key is tied to a **domain name**: an IP address does not work.
Devices	Modern browsers (Chrome, Edge, Safari, Firefox) with a fingerprint, PIN or screen lock configured. It does not work in Chrome for Android incognito mode.

2. Process overview

Adding the template to an application takes nine steps, described in chapter 4:

1. Install the template in Clarion (done by the installer, chapter 3) and check that it is registered.
2. Check that `NetTalk_ARMiLogin.js` was copied to the executable's `web\scripts` folder (done by the template when generating, see 3.2).
3. Add the **global** extension to the application.

4. Add the **Form** extension to the LoginForm procedure (NetWebForm) and fill in its prompts.
5. Write the **session** code in the `ARMi Login: fingerprint verified` embed.
6. Add the **Offer** extension to the destination page that follows the login.
7. Review or translate the texts of the extensions.
8. Generate and compile the application.
9. Test the fingerprint sign-in.

2.1 The passkey code: how the data travels

This is what is most worth understanding before configuring the extensions. When the fingerprint is activated, the server stores with it a value that **you choose**: the passkey code. Later, when someone signs in with that fingerprint, the server hands it back to your login. It can be any data that identifies the user: the user name, the e-mail or an internal id (for example 10015). It does not have to be what the user types in the login.

When	Where it is configured	What happens to the code
Activation: the signed-in user accepts "Activate fingerprint"	Offer extension, prompt **Value to store in the passkey**	The expression is evaluated (for example `p_web.GetSessionValue('LOGIN_ID')`) and the result is stored with the fingerprint in NetTalkPasskeys.tps (UserKey field).
Fingerprint sign-in: the user taps "Login with fingerprint"	Form extension, prompt **Variable to receive the passkey code**	The server finds the fingerprint, takes the stored code and puts it in that variable (for example `Loc:Login`), and also as `p_web.GetValue('loc:login')` and as the session value loc:login.
After the sign-in	Embed **ARMi Login: fingerprint verified** (in the login procedure)	Your code uses `p_web.GetValue('loc:login')` to find the user and load the session: the same code the normal login runs when OK is pressed, without the password comparison.

Example: if your normal login identifies the user by an internal id, store that id (for example 10015) when activating the fingerprint and, in the embed, look the user up by id using `p_web.GetValue('loc:login')`. If it identifies the user by e-mail, store the e-mail. All that matters is that the stored data is the same data the embed looks up.

- The value must be unique for each user and cannot be empty.
- It should be stable: if a user's data changes (for example the e-mail), fingerprints already activated keep returning the old value; they must be deleted and activated again (see the FAQ about this case).
- In the embed the value is read with `p_web.GetValue('loc:login')` (the chosen variable name, in lower case). The form's local variable only lives while the login OK is processed.

> **Important:** **Both prompts refer to the same data:** what you enter in **Value to store in the passkey** (Offer extension) is what you receive in **Variable to receive the passkey code** (Form extension). If that data is not what your embed expects to find the user, the fingerprint sign-in will not find them.

3. What the installer does

The installer copies the template files to the Clarion folders and makes the template available in the IDE. **It does not modify any application (.app), dictionary or other templates.** This chapter details exactly what it writes and where.

3.1 Files that are copied

File	Destination folder	What it is for
`NetTalk_ARMiLogin.inc`	`<Clarion>\Accessory\LibSrc\Win\`	Declaration of the `NetWebAuthnClass` class.
`NetTalk_ARMiLogin.clw`	`<Clarion>\Accessory\LibSrc\Win\`	Implementation of the class (challenges, signature verification with CNG, credential store).
`NetTalk_ARMiLogin.tpl`	`<Clarion>\Accessory\Template\Win\`	Template header; this is the file registered in the IDE.
`NetTalk_ARMiLogin.tpw`	`<Clarion>\Accessory\Template\Win\`	The three extensions and their prompts.
`bcrypt.lib`	`<Clarion>\Accessory\Lib\`	Import library for `bcrypt.dll`. Without it the program does not link.
`NetTalk_ARMiLogin.js`	`<Clarion>\Accessory\LibSrc\Win\`	Browser script (master copy). **It is not installed inside any application**: the template copies it to the `web\scripts` folder of each app's executable when the app is generated (see 3.2).
This manual (.docx / .pdf)	`<Clarion>\ARMi\Nettalk_ARMiLogin`	Documentation.
Sample application	`<Clarion>\ARMi\Nettalk_ARMiLogin`	See chapter 8.

The `Accessory\LibSrc\Win`, `Accessory\Template\Win` and `Accessory\Lib` folders are part of the standard Clarion search paths (redirection `.red` file), so the compiler, the linker and the IDE find the files without any configuration. Verified: the linker finds `bcrypt.lib` in `Accessory\Lib`. On the other hand, a copy in the Clarion root folder (`<Clarion>\bcrypt.lib`) is **not** found.

3.2 What the template does when the application is generated

The developer does not have to copy anything by hand. Every time the application is **generated** (Generate, or Make when the IDE regenerates before compiling) the template runs these actions, in the same way NetTalk copies its own web files:

1. It checks that `bcrypt.lib` exists in `<Clarion>\Accessory\Lib` (or in `<Clarion>\Lib`). If it is missing, it shows an error in the IDE: "bcrypt.lib not found ... Reinstall NetTalk ARMi Login".
2. It looks for `NetTalk_ARMiLogin.js` in `<Clarion>\Accessory\LibSrc\Win`. If it is missing, it shows an error.
3. It copies the script to the `scripts` folder inside the web folder of the executable. The web folder name is the one NetTalk uses (the **(To) Web Folder** prompt of its global extension, `web` by default),

placed relative to the folder where the Clarion redirection file writes the executable (for example `.\Exe`), which is where the server looks for it at run time. If `scripts` does not exist, it creates it.

4. If **Do not copy Web Folder** is checked in NetTalk, it copies nothing and warns in the IDE that the script must be copied by hand.

To copy the file, the template creates a temporary `ARMiLoginCopy.Bat` file (the Clarion redirection file decides where it goes, for example `.\tmp`), runs it by its full path and deletes it. When it finishes it writes this to the IDE message window: `NetTalk ARMi Login: NetTalk_ARMiLogin.js copied to web\scripts`.

> **Note:** `bcrypt.lib` is not copied: Clarion already searches for `.lib` files in `Accessory\Lib`, and the template adds `bcrypt.lib` to the project libraries. All it does with it is check that it is there.

> **Important:** The copy is made when generating. If the installer is updated (a new `.js`), each application must be regenerated to receive the new version.

3.3 What the installer does NOT touch

- Any `.app` application, `.txa`, `.cwproj` project or `.dct` dictionary.
- The NetTalk, StringTheory or Clarion templates and classes.
- Any application's web folder or its data.

3.4 What the template and the application write afterwards

Once the developer uses the template in an application, the following happens (it is not done by the installer):

What	Where	Detail
Library in the project	The application's `.cwproj` file	When generating, the template adds `bcrypt.lib` to the library list.
Script in the application	`<exe folder>\web\scripts\NetTalk_ARMiLogin.js`	Copied by the template when generating (see 3.2). The temporary `ARMiLoginCopy.Bat` file is deleted when it finishes.
Class `INCLUDE`	Generated code (main `.clw`)	`INCLUDE('NetTalk_ARMiLogin.inc'),ONCE`
`WAAuth` object	Local data of each procedure with the extension	One instance per request: it is not shared between threads.
Credentials	`NetTalkPasskeys.tps` (name is configurable)	Created by the server on the first activation, in the folder where the `.exe` runs.
Browser marks	The site's `localStorage`	`netwebauthn_known` (this device has already activated the fingerprint) and `netwebauthn_declined` (the user chose "do not ask again").
Temporary challenge	NetTalk session	`_wa_chal_` (single use) and `_wa_done_` (internal mark that the reply was already sent).
Application log	`p_web.AddLog`	`fingerprint login <user>`, `fingerprint login refused: <reason>`, `fingerprint activated for <user>`.

Contents of NetTalkPasskeys.tps

One record for each activated fingerprint. **It only contains public keys**; if someone took the file they could not sign in with it.

Field	Type	Contents
`CredId`	STRING(256)	Credential identifier (unique key). The device sends it when signing in.
`UserKey`	STRING(255)	Passkey code saved with the fingerprint; it identifies the user (what the login receives back in its variable).
`PublicKey`	STRING(160)	ES256 public key in base64url, used to verify the signature.
`SignCount`	ULONG	Signature counter; detects cloned or replayed credentials.
`Device`	STRING(60)	Device platform (informational).
`Created`, `CreatedTime`	LONG	Date and time the fingerprint was activated.
`LastUsed`	LONG	Date of the last sign-in.

Keys: `ByCred` (primary key, unique) and `ByUser` (allows duplicates: a user can have several fingerprints, one per device).

4. Adding the template to a NetTalk Server application

> **Note:** The screenshots correspond to Clarion 11.1. In other IDE versions the menus may differ slightly. The exact names of the extensions are those in the table in chapter 5.

Step 1. Check that the template is registered

1. Open the Clarion IDE and choose **Setup > Template Registry**.
2. Look for **NetTalk ARMi Login** in the list. If it does not appear, use **Register...** and choose `<Clarion>\Accessory\Template\Win\NetTalk_ARMiLogin.tpl`.
3. Close the window with **Close**.

Step 2. The browser script

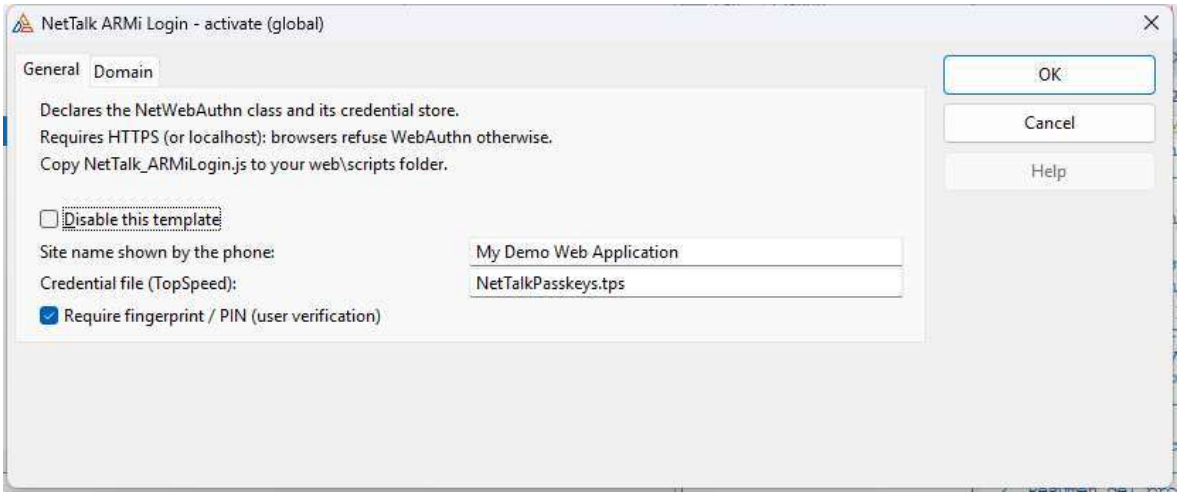
Nothing has to be copied by hand: the template copies `NetTalk_ARMiLogin.js` to the executable's `web\scripts` folder every time it is generated (see 3.2). This step is only a **check**, done after generating for the first time (step 8): the file must be next to `netweb.js`, in the folder the NetTalk server publishes as `/scripts/`.

> **Note:** To check it with the server running, open `http://localhost:<port>/scripts/NetTalk_ARMiLogin.js` in the browser: it must show the script code and not a 404 error.

> **Important:** If **Do not copy Web Folder** is checked in NetTalk, the template does not copy the script: copy it by hand from `<Clarion>\Accessory\LibSrc\Win`.

Step 3. Add the global extension

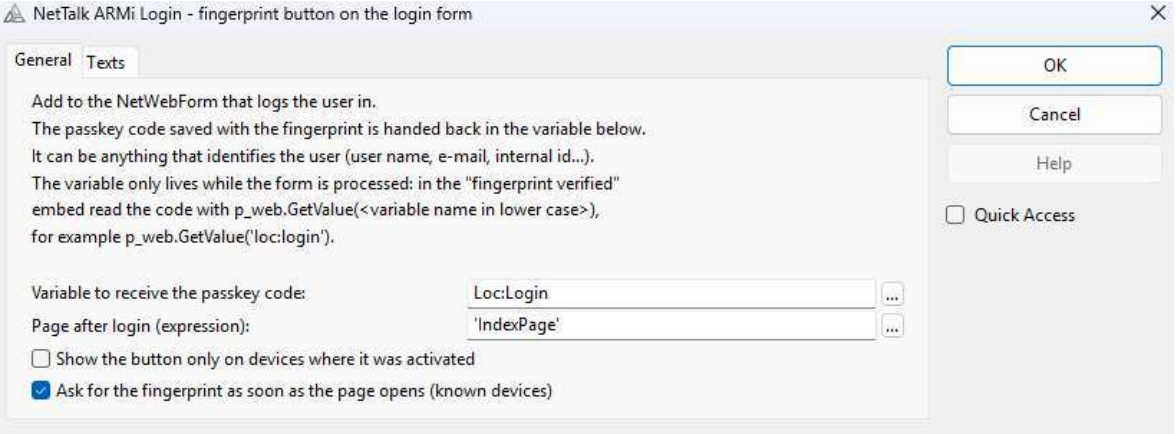
1. Open the application in the IDE (application tree).
2. Go to **Global Properties** and press the **Extensions** button.
3. Press **Insert** and choose **NetTalk ARMi Login - activate (global)**.
4. Fill in the **General** tab: the site name (what the user will see in their device's window) and the credential file.
5. Leave the **Domain** tab empty, unless the server is behind a proxy (see chapter 7).
6. Accept with **OK**.



Step 4. Login procedure extension

This is the NetWebForm procedure that shows user name and password (in the sample applications it is called `LoginForm`).

1. In the procedure list, open the login procedure.
2. Press the **Extensions** button and then **Insert**.
3. Choose **NetTalk ARMi Login - fingerprint button on the login form**.
4. In **Variable to receive the passkey code** choose the variable that receives the code saved with the fingerprint (for example `Loc:Login`). That code can be any data that identifies the user (name, e-mail, an internal id). The variable only exists while the form processes the login OK: in the step 5 embed the code is read with `p_web.GetValue('loc:login')`.
5. In **Page after login** type the destination page **in quotes**, for example `'IndexPage'`. It is an expression.
6. On the **Texts** tab review or translate the texts (see step 7).
7. Accept with **OK**.



Step 5. User session code (embed)

When the fingerprint is valid, the server puts the code saved with the fingerprint in the variable chosen in step 4, marks the session as logged in (`p_web.ValidateLogin()`) and runs an **embed** so that the developer can load that user's session data. The code is also available as a web value and a session value, named like the variable in lower case (for example `p_web.GetValue('loc:login')`). It is better not to use the form's local variable inside the embed.

1. Open the **Embeditor** of the login procedure.
2. Find the **ARMi Login: fingerprint verified** embed, under **Routines > ARMi Login**.
3. Write there the same code that the normal login runs after validating the password (session variables, access level, last sign-in date, etc.).

Generic example (each application loads its own session variables):

```
! The code saved with the fingerprint (the one that identifies the user) is in
p_web.GetValue('loc:login').
! p_web.ValidateLogin() was already called by the template.
p_web.SetSessionValue('LOGIN_ID',clip(p_web.GetValue('loc:login')))
p_web.SetSessionValue('LOGIN_name',clip(MyTable:Name))
p_web.SetSessionLevel(0)
p_web.AddLog('fingerprint login OK: ' & clip(p_web.GetValue('loc:login')))
```

> **Important:** The code in this embed **must not compare passwords**: the identity was already verified by the device's signature. If the normal login looks the user up in a table (for example by e-mail), do the same lookup here using `p_web.GetValue('loc:login')`. Note that the code is the data that was saved when the fingerprint was activated (it can be an e-mail, a name or an id): the lookup must use that same data. Do not use the form's local variable (`Loc:Login`): it only exists while the login form is being processed.

Step 6. Offer extension on the destination page

This is the page the user goes to after signing in (for example `IndexPage`). It shows, once per device, the "Activate fingerprint?" window and handles the activation.

1. Open the destination page procedure (NetWebPage or NetWebForm).
2. Press **Extensions > Insert** and choose **NetTalk ARMi Login - offer to activate the fingerprint after login**.
3. In **Show the offer when** leave `p_web.GetSessionLoggedIn() = 1` (the offer is only shown to users who are already logged in).
4. In **Value to store in the passkey** type the expression that returns the data that identifies the user (the data the fingerprint login will receive back), for example `p_web.GetSessionValue('LOGIN_ID')`.

5. Accept with **OK**.



> **Important:** **The code saved with the fingerprint is what the login receives back, so it must be the data the embed uses to find the user.** If the activation saves `john@mail.com` and the embed looks the user up by name (`JOHN`), the fingerprint sign-in will not find the user. Always use the same data in both places.

Step 7. Texts and translation

All the texts the user sees are on the **Texts** tab of each extension and go through `p_web.Translate()`. There are two ways to have them in the application's language:

- Type them directly in the prompt (simplest if the application has a single language).
- Load each English text with its translation in NetTalk's translation table (if the application is multi-language).

Step 8. Generate and compile

1. Generate the application (**Generate** or **Make**).
2. The IDE message window must show `NetTalk ARMi Login: NetTalk_ARMiLogin.js copied to ...` (step 2). If an error about `bcrypt.lib` or the `.js` appears, the installation is incomplete (see 3.1).
3. Compile. If the linker reports `Unresolved External BCrypt...`, `bcrypt.lib` is missing from `<Clarion>\Accessory\Lib`.
4. Copy the executable and the `web` folder (with the `.js`) to the place where it runs.

Step 9. Test

Do the first test on the same PC, with `http://localhost:<port>` (it is a secure origin).

1. Open the login page. The **Login with fingerprint** button must appear to the right of the login button.
2. Tap the button: since the device has not activated the fingerprint yet, a "not activated" notice appears. Close it with **OK**.
3. Sign in with user name and password. On the destination page the modal window **Activate fingerprint login on this device?** appears. Choose **Activate** and confirm with Windows Hello (PIN or fingerprint).
4. Sign out and tap **Login with fingerprint**. Windows shows the site's key; confirm. You are taken straight to the destination page.

To repeat the test from scratch: delete `NetTalkPasskeys.tps` on the server and run `localStorage.clear()` in the browser console (F12 > Console).

> **Note:** To test from a phone you need HTTPS with a domain name. During development you can use a tunnel (for example ngrok) to the local server port, remembering that it **exposes the server to the internet while it is open**.

Variant: Multi-DLL program

In a Multi-DLL program the NetWebAuthn class is compiled and exported by **one** DLL (the **root DLL**); every other DLL and the EXE import it from there. This is set up in the **Multi-DLL** tab of the global extension. A program made of a single EXE needs nothing here: leave the tab as it is.

1. Choose the root DLL: the one that will hold the class (usually the one that already holds the common classes).
2. Add the **global** extension to **every** app of the program (root DLL, the other DLLs and the EXE), as in step 3, and fill in the **General** tab in the same way in all of them.
3. In each app open the **Multi-DLL** tab and tick **This is part of a Multi-DLL program**. In a DLL it is ticked automatically; **in the EXE you must tick it yourself**.
4. Only in the root DLL tick **Export the NetWebAuthn class from this DLL**. In all the other apps leave it unticked.
5. Tick **Copy NetTalk_ARMiLogin.js to the web folder when generating** only in the app that runs the NetTalk web server (normally the EXE). In the DLLs it is unticked by default.
6. Add the **Form** and **Offer** extensions to the procedures where they belong, in whichever app holds them (steps 4 to 6).
7. Generate and compile the root DLL first, then the other DLLs and the EXE.

> **Note:** `bcrypt.lib` is linked only by the root DLL (or by the single app in a normal program), so the other apps do not need it.

> **Important:** Tick **Export the NetWebAuthn class from this DLL** in **one** DLL only. If it is ticked in two, the linker reports duplicate exports; if it is ticked in none, the EXE reports **Unresolved External** errors on `NetWebAuthnClass` methods.

5. Extension reference

5.1 NetTalk ARMi Login - activate (global)

Added once per application, in the global properties.

Prompt	Type	Default value	Description
Disable this template	Check box	No	Disables all code generation of the template in this application.
Site name shown by the phone	Text	`My Web Application`	Site name that the device shows when creating the key. Required.
Credential file (TopSpeed)	Text	`NetTalkPasskeys.tps`	Credential file. It can include a full path. Required.
Require fingerprint / PIN	Check box	Yes	Requires the user to verify on the device (fingerprint, PIN). Leave it checked.
RP id (domain)	Expression	empty	Domain of the key. Empty = taken from each request's Host, without the port.

Prompt	Type	Default value	Description
Expected origin	Expression	empty	Expected origin. Empty = `https://` + Host (or `http://` if it is localhost).
This is part of a Multi-DLL program	Check box	Yes in a DLL, No in an EXE	**Multi-DLL tab.** Tells the template that the program is Multi-DLL. Tick it in every app of the program, EXE included.
Export the NetWebAuthn class from this DLL	Check box	No	**Multi-DLL tab.** Only in the root DLL: it compiles the class and exports it. The other apps import it.
Copy NetTalk_ARMiLogin.js to the web folder when generating	Check box	Yes in an EXE, No in a DLL	**Multi-DLL tab.** Leave it ticked only in the app that runs the NetTalk web server (see 3.2).

5.2 NetTalk ARMi Login - fingerprint button on the login form

Added to the login NetWebForm procedure. Requires the global extension.

Prompt	Type	Default value	Description
Variable to receive the passkey code	Field	(required)	Variable that receives the code saved with the fingerprint (e.g. `Loc:Login`). It only lives while the form is processed: in the embed read the code with `p_web.GetValue('loc:login')`.
Page after login	Expression	`'IndexPage'`	Page to redirect to after signing in. Goes in quotes.
Show the button only on devices where it was activated	Check box	No	Hides the button on devices without the activation mark.
Ask for the fingerprint as soon as the page opens	Check box	No	On devices that already activated it, starts the verification when the page loads.
Button	Text	`Login with fingerprint`	Button text.
Login failed	Text	`The fingerprint could not be verified. Use your user and password.`	Message when the server rejects the sign-in.
Not supported	Text	`This browser or connection does not support fingerprint login (HTTPS is required).`	Browser or connection without WebAuthn.

Prompt	Type	Default value	Description
Not activated yet	Text	`Fingerprint login is not activated on this device yet. Log in with your user and password and you will be offered to activate it.`	Notice when the button is tapped on a device that has not activated it.
Not registered on the server	Text	`This fingerprint is no longer registered on the server. Log in with your user and password and activate it again.`	The device has a key that the server no longer knows.
Button OK / Button try anyway	Text	`OK` / `Try anyway`	Buttons of the "not activated" notice.

Embed it adds: **ARMi Login: fingerprint verified** (**Routines > ARMi Login**).

5.3 NetTalk ARMi Login - offer to activate the fingerprint after login

Added to the destination page that follows the login (NetWebPage or NetWebForm). Requires the global extension.

Prompt	Type	Default value	Description
Show the offer when	Expression	`p_web.GetSessionLoggedIn() = 1`	Condition to show the offer.
Value to store in the passkey	Expression	`p_web.GetSessionValue('loc:login')`	Code saved with the fingerprint; it identifies the user. The fingerprint login returns it in the step 4 variable (`p_web.GetValue('loc:login')`).
Question / Yes / Later / Never	Text	`Activate fingerprint login on this device?` / `Activate` / `Not now` / `Do not ask again`	Texts of the modal offer window.
Done / Failed	Text	`Fingerprint login activated.` / `The fingerprint could not be activated.`	Result of the activation.
Already activated	Text	`Fingerprint login is already activated on this device.`	Notice when this device already had the fingerprint activated (the browser does not allow creating a duplicate key).
Not supported	Text	`This browser or connection does not support fingerprint login.`	Browser without WebAuthn.

6. How it works internally

6.1 Activating a fingerprint

1. The user signs in with user name and password and reaches the destination page.
2. The script sees that the device is not marked as activated and shows the offer window.
3. When accepted, the browser asks the server for a challenge (`_wa_=ropts`).
4. The device creates a key pair (ES256) and signs with the fingerprint or PIN. The private key stays on the device.
5. The browser sends the server the public key and the signed data (`_wa_=rverify`).
6. The server verifies the challenge, the origin, the domain and the flags, and stores the public key in `NetTalkPasskeys.tps`.

6.2 Signing in with the fingerprint

1. The user taps **Login with fingerprint**. The browser asks the server for a challenge (`_wa_=lopts`).
2. The device shows the site's keys and the user confirms with fingerprint or PIN. The device signs the challenge.
3. The browser sends the signature (`_wa_=lverify`).
4. The server looks up the credential, verifies the signature with the public key, checks that the counter increased and loads the saved code into the login variable.
5. It calls `p_web.ValidateLogin()`, runs the step 5 embed and replies with the destination page; the browser redirects.

6.3 What the server verifies

- The **type** of operation (`webauthn.create` / `webauthn.get`) and that the **challenge** matches the one issued in that session (single use).
- That the **origin** and the **domain** hash correspond to this application.
- That the user was present and was **verified** (fingerprint or PIN), if the prompt requires it.
- The ES256 **signature** with the stored public key.
- That the **signature counter** increases (devices that always report 0 are accepted).

6.4 Browser - server communication

The script talks to the same page that has the extension, adding the _wa_ parameter. The replies are small JSON objects (`{"ok":1,...}`):

Parameter	Handled in	Reply
`_wa_=lopts`	Login procedure	Challenge and domain for the sign-in.
`_wa_=lverify`	Login procedure	`{"ok":1,"next":...}` if the sign-in is valid; `err` = `auth` or `unknown` if not.
`_wa_=ropts`	Destination page	Challenge, user data and already registered credentials (to avoid duplicates).
`_wa_=rverify`	Destination page	`{"ok":1}` if the activation was saved.

7. Going to production

Before publishing, the server must be reachable over **HTTPS with a valid certificate** and a **domain name**.

1. Compile the application on the development PC (where `bcrypt.lib` is) and copy the new `.exe` to the server.
2. Deploy the executable's `web` folder: it already contains `web\scripts\NetTalk_ARMiLogin.js`, copied by the template when generating. If compressed (`.gz`) versions of the scripts are used, generate them again.
3. **Do not copy or overwrite** `NetTalkPasskeys.tps`. It is production data, like the other `.tps` files. If it already exists it is kept; if it does not exist, the server creates it on the first activation. Include it in the backup.
4. Check that the folder where the `.exe` runs allows writing (the `.tps` is created there), or specify another path in the **Credential file** prompt.
5. Test with a real phone: sign in with the password, activate, sign out and sign in with the button.

> **Note:** If the credential file is lost, nobody is locked out: each user signs in with their password and activates the fingerprint again. The script detects that the device has a key the server no longer knows and offers the activation again.

7.1 Server behind a proxy or forwarding

If IIS (ARR), a load balancer or a tunnel changes the `Host` header before it reaches NetTalk, the server will not know the public domain. In that case, on the **Domain** tab of the global extension fill in:

Prompt	Example
RP id (domain)	`'portit.com.ar'` (the public domain, without `https://` or port)
Expected origin	`'https://portit.com.ar'` (with `https://`; if it uses a port other than 443, add it)

> **Important:** Keys are tied to the domain. Keys created on `localhost` or on a test tunnel **do not work in production**: each user must activate the fingerprint again on the real domain. Changing the domain requires a new activation on every device.

8. Sample application

A minimal application that already has the three extensions configured is delivered together with the template, so the developer can see the result before modifying their own application.

Item	Value
Application name	NtLoginDemo.app
Folder where the installer puts it	`<Clarion>\ARMi\Nettalk_ARMiLogin`
Login procedure	LoginForm
Destination page	IndexPage
Test user and password	demo / demo

8.1 How to use it

1. Open the sample application in the IDE and generate / compile (it requires `bcrypt.lib`).
2. Run the server and open `http://localhost:<port>` in the browser.
3. Follow step 9 of chapter 4 to activate the fingerprint and sign in with it again.
4. Review in the IDE how the three extensions and the embed are configured, and repeat it in your own application.

9. Troubleshooting

Symptom	Likely cause and solution
It does not compile: `Unresolved External BCrypt...`	`bcrypt.lib` is missing from `<Clarion>\Accessory\Lib`. A copy in the Clarion root folder is not found by the linker.
When generating, "bcrypt.lib not found" or "NetTalk_ARMiLogin.js not found" appears	Incomplete installation: reinstall NetTalk ARMi Login (see 3.1).
Strange errors when compiling the `.clw` or `.inc` ("Illegal data type", "Field not found")	Clarion files must have **CRLF** line endings. If they were edited with a tool that saves only LF, the compiler reads them as a single line. Save them again with CRLF.
The button does not appear	The page is not a secure origin (use HTTPS or `http://localhost`; with `http://IP` or `http://pc-name` the browser does not allow WebAuthn). Or the browser loaded an old copy of the `.js` (reload with Ctrl+F5). Or `NetTalk_ARMiLogin.js` is missing from `web\scripts` (regenerate the application so the template copies it).
The button appears but activating gives a domain error	An IP address is being used as the address. The key needs a domain name.
"Activate" does nothing	Chrome incognito mode (it does not allow creating passkeys on the device): use a normal tab. Or the device has no screen lock / Windows Hello configured.
"no longer registered on the server" appears	The server does not have the credential (file deleted or different). Sign in with the password and activate again. Check that the server always reads the same `.tps`.
With a proxy / tunnel the origin or domain fails	Fill in **RP id** and **Expected origin** on the Domain tab of the global extension (see 7.1).
Sign-in works but session data is missing	The `ARMi Login: fingerprint verified` embed is empty or does not load the same variables as the normal login.
The sign-in is rejected and it is not clear why	Check the application log: `fingerprint login refused: <reason>`. To see the browser's error name, set `debug: true` in the `var cfg = {...}` line of `NetTalk_ARMiLogin.js` (only while testing).

9.1 Class error codes

The reason for the last rejection is in `WAAuth.LastError` (code) and `WAAuth.LastErrorText` (text).

Code	Constant	Meaning
0	`WA:Ok`	Operation successful.
1	`WA:Err:BadInput`	Incomplete or malformed data.
2	`WA:Err:ClientData`	Type, challenge or origin do not match.
3	`WA:Err:RpIdHash`	The credential's domain is not this application's.
4	`WA:Err:Flags`	User presence or user verification is missing.
5	`WA:Err:Unknown`	The credential is not registered on the server.
6	`WA:Err:Signature`	The signature does not verify.
7	`WA:Err:Counter`	The signature counter did not increase (possible clone or replay).
8	`WA:Err:Crypto`	A call to `bcrypt.dll` failed.
9	`WA:Err:Store`	The credential file could not be opened or written.

10. Frequently asked questions

What happens if the user changes phones?

They sign in with their password on the new phone and activate the fingerprint there. A user can have several fingerprints activated (one per device); the previous ones keep working.

What happens if the data that identifies the user changes?

Fingerprints already activated keep the value that was stored when they were activated (for example the old e-mail), so the fingerprint sign-in will return that old value. Delete that user's fingerprints (`DeleteCredentials` method) and ask them to activate the fingerprint again. That is why it is best to store data that does not change, such as an internal id.

Can a fingerprint be removed?

The class includes the `DeleteCredentials(user)` method, which deletes all of a user's fingerprints. The template does not yet include a screen for it; it can be called from your own administration procedure.

Does it work with synced keys (iCloud, Google)?

Yes. If the user has the key synced on another device but this browser never used it, the button shows the "not activated" notice with the **Try anyway** option, which lets them try with the synced key.

Is it safe to store the credentials in a .tps?

The file contains only public keys, counters and informational data: it cannot be used to sign in. Even so, protect it like the rest of the application's data and keep a backup.

Can it be used without the "Offer" page?

No: it is the one that offers and registers the fingerprint. Without it no user could activate it.

Is there another way to store the credentials besides a .tps?

Not today. Storing them in a table of the application's database is planned as a future improvement.

Appendix A. Glossary

Term	Meaning
WebAuthn	W3C standard that lets a website verify the user with a device authenticator (fingerprint, face, PIN).
Passkey	A WebAuthn credential stored on the device or synced by the platform. It replaces the password.
RP id	Identifier of the site ("relying party"): the domain name the key is tied to.
Origin	Scheme + domain + port of the page (for example `https://portit.com.ar`).
Challenge	Single-use random value that the server issues and the device signs, so that a sign-in cannot be replayed.
ES256	ECDSA signature with the P-256 curve and SHA-256. It is the algorithm this template uses.
CNG	Cryptography API: Next Generation, the Windows cryptographic API (`bcrypt.dll`).
User verification (UV)	Local confirmation of the person on the device: fingerprint, face or PIN.